

Day 9 — Aggregation

COUNT, SUM, GROUP BY & HAVING — Notes for Recap

All examples continue using the `online_retail` table from Day 8: `invoice_no`, `stock_code`, `description`, `quantity`, `invoice_date`, `unit_price`, `customer_id`, `country`.

1. COUNT / SUM / AVG / MIN / MAX

Aggregate functions take many rows and collapse them into a single summary value - the SQL equivalent of pandas' `.sum()`, `.mean()` etc. from Day 5.

```
SELECT COUNT(*) FROM online_retail;           -- total number of rows
SELECT COUNT(DISTINCT customer_id) FROM online_retail; -- unique customers

SELECT SUM(quantity * unit_price) FROM online_retail; -- total revenue
SELECT AVG(unit_price) FROM online_retail;           -- average price
SELECT MIN(unit_price), MAX(unit_price) FROM online_retail;
```

Function	What it Returns
COUNT()	Number of rows (or non-null values in a column)
SUM()	Total of a numeric column
AVG()	Average of a numeric column
MIN() / MAX()	Smallest / largest value in a column

Key takeaway: `COUNT(*)` counts all rows; `COUNT(column)` counts only non-NULL values in that column - these can give different results when a column has missing data, exactly like `customer_id` in this dataset.

2. GROUP BY

`GROUP BY` splits rows into groups based on one or more columns, then applies an aggregate function to each group separately - the SQL equivalent of pandas' `groupby()` from Day 5.

```
-- Total revenue per country
SELECT
    country,
    SUM(quantity * unit_price) AS total_revenue
FROM online_retail
GROUP BY country;

-- Number of orders and average order value per customer
SELECT
    customer_id,
    COUNT(*) AS num_orders,
    AVG(quantity * unit_price) AS avg_order_value
FROM online_retail
GROUP BY customer_id;
```

Key takeaway: Every column in the SELECT list that isn't inside an aggregate function must also appear in GROUP BY - this is one of the most common SQL errors for beginners.

3. HAVING

HAVING filters groups *after* aggregation - it's like WHERE, but for aggregated results instead of individual rows.

```
-- Countries with total revenue over 100,000
SELECT
    country,
    SUM(quantity * unit_price) AS total_revenue
FROM online_retail
GROUP BY country
HAVING SUM(quantity * unit_price) > 100000;

-- Customers who placed more than 20 orders
SELECT
    customer_id,
    COUNT(*) AS num_orders
FROM online_retail
GROUP BY customer_id
HAVING COUNT(*) > 20;
```

Key takeaway: WHERE filters rows before grouping; HAVING filters groups after aggregation. You cannot use an aggregate function like SUM() inside WHERE - that's exactly what HAVING exists for.

4. Practice: Common Business Questions

Top customers by revenue

```
SELECT
    customer_id,
    SUM(quantity * unit_price) AS total_spent
FROM online_retail
WHERE customer_id IS NOT NULL
GROUP BY customer_id
ORDER BY total_spent DESC
LIMIT 10;
```

Monthly revenue trend

```
SELECT
    DATE_TRUNC('month', invoice_date) AS month,
    SUM(quantity * unit_price) AS monthly_revenue
FROM online_retail
GROUP BY DATE_TRUNC('month', invoice_date)
ORDER BY month;
```

Average order value (AOV)

```
SELECT
    AVG(order_total) AS average_order_value
FROM (
    SELECT invoice_no, SUM(quantity * unit_price) AS order_total
    FROM online_retail
    GROUP BY invoice_no
) AS order_totals;
```

Most profitable products

```
SELECT
    description,
    SUM(quantity * unit_price) AS total_revenue
FROM online_retail
GROUP BY description
ORDER BY total_revenue DESC
LIMIT 10;
```

Key takeaway: Notice that Average Order Value needed a subquery - first totaling each invoice, then averaging those totals. This is a common real-world pattern: aggregate once to get per-group totals, then aggregate again on top of that result.

Day 9 Summary — Aggregation

Concept	One-Line Recap
COUNT/SUM/AVG/MIN/MAX	Collapse many rows into a single summary value

GROUP BY	Split rows into groups before aggregating - like pandas groupby()
HAVING	Filter groups after aggregation (WHERE filters rows before)
Top N queries	GROUP BY + ORDER BY + LIMIT answers most 'top customers/products' questions
Nested aggregation	Aggregate once, then aggregate again on top (e.g. Average Order Value)

Overall takeaway: Aggregation is where SQL starts answering real business questions instead of just retrieving rows. GROUP BY + aggregate functions + HAVING is the exact pattern used behind almost every dashboard metric and KPI you've ever seen.